

Metodologie di Programmazione (A-L)

II semestre - a.a. 2025 – 2026

Stream I

a cura di Massimo La Morgia



SAPIENZA
UNIVERSITÀ DI ROMA

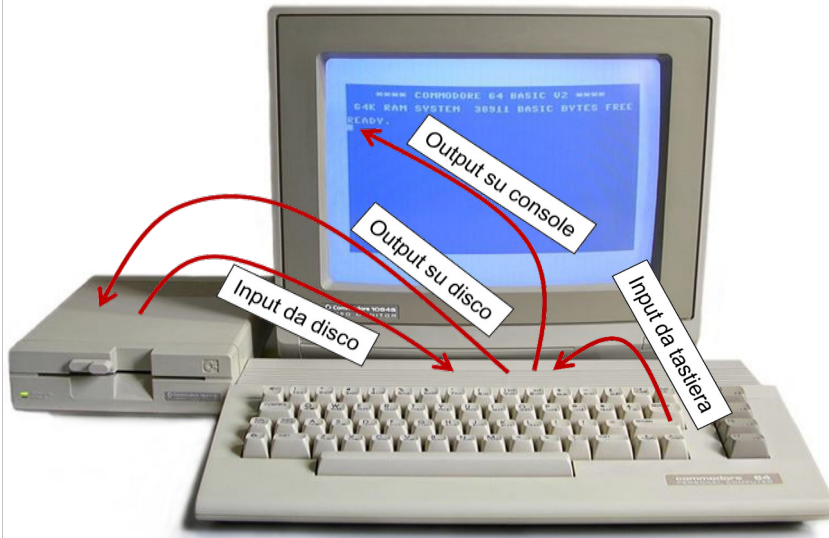
*Tutti i diritti relativi al presente materiale didattico ed al suo contenuto sono riservati a Sapienza e ai suoi autori (o docenti che lo hanno prodotto). È consentito l'uso personale dello stesso da parte dello studente a fini di studio. Ne è vietata nel modo più assoluto la diffusione, duplicazione, cessione, trasmissione, distribuzione a terzi o al pubblico pena le sanzioni applicabili per legge.

**I crediti sulle slide di questo corso sono riportati nell'ultima slide

Stream<T> parte I

Input & Output

Stream<T>



Gli **Stream** di I/O sono un concetto ben distinto da **Stream API**.

java.util.Optional come contenitore di riferimenti

- java.util.Optional è un contenitore di un riferimento che potrebbe essere o non essere null
- Un metodo può restituire un Optional invece di restituire un riferimento potenzialmente null
- Serve a evitare i NullPointerException

Se non avessimo gli optional, dovremmo tornare null

```
public static String trovaNome(boolean trovato) {  
    if (trovato) {  
        return "Mario";  
    }  
    return null;  
}
```

Con gli Optional possiamo tornare un valore "vuoto"

```
public static Optional<String> trovaNome(boolean trovato) {  
    if (trovato) {  
        return Optional.of("Mario");  
    }  
    return Optional.empty();  
}
```

Creare e verificare un `java.util.Optional`

- Un `Optional` senza riferimento (contenitore vuoto):

```
Optional.empty()
```

- Un `Optional` non nullo:

```
Optional.of("ciao mondo");
```

- Un `Optional` di un riferimento che può essere nullo:

```
Optional<String> optional = Optional.ofNullable(s);
```

- Controllo della presenza di un valore non null:

```
Optional.empty().isPresent() == false
```

```
Optional.of("ciao mondo").isPresent() == true
```

Ottenere il valore di un java.util.Optional

- Mediante `orElse`

```
Optional<String> op = Optional.of("ecco mi")
```

```
String s1=op.orElse("fallback"); // "ecco mi"
```

```
String s2= Optional.empty().orElse("fallback"); // "fallback"
```

- Mediante `ifPresent` o `ifPresentOrElse`:

```
op.ifPresent(System.out::println); // "ecco mi"
```

- Da **non** usare `(! ? # @)`:

```
optional.get(); // valore o solleva eccezione se non presente
```

Stream<T>

Un nuovo meccanismo per le sequenze di elementi: gli Stream

- Una nuova interfaccia `java.util.stream.Stream`
- Rappresenta una sequenza di elementi su cui possono essere effettuate una o più operazioni
- Supporta operazioni sequenziali e parallele
- Uno `Stream` viene creato a partire da una sorgente di dati, ad esempio una `java.util.Collection`
- Al contrario delle `Collection`, uno `Stream` non memorizza né modifica i dati della sorgente, ma opera su di essi

Stream: operazioni intermedie e terminali (1)

- Le operazioni possono essere **intermedie** o **terminali**
 - **Intermedie**: restituiscono un altro stream su cui continuare a lavorare
 - **Terminali**: restituiscono il tipo atteso

[4, 8, 15, 16, 23, 42] $\xrightarrow{\text{stream()}}$ $\xrightarrow{\text{filter}(x \rightarrow x > 15)}$ $\xrightarrow{\text{count()}}$ 3

- Una volta che uno stream è stato consumato (**operazione terminale**), non può essere riutilizzato
- **Builder pattern** (che vedremo poi): si impostano una serie di operazioni per configurare (op. intermedie) e infine si costruisce l'oggetto (op. terminale)

Metodi principali dell'interfaccia `java.util.stream.Stream<T>`

Metodo	Tipo	Descrizione
<code>Stream<T> filter(Predicate<? super T> predicate)</code>	I	Fornisce uno stream che contiene solo gli elementi che soddisfano il predicato
<code><R> Stream<R> map(Function<? super T, ? extends R> mapFunction)</code>	I	Applica la funzione a tutti gli elementi dello stream, fornendo un nuovo stream di elementi mappati
<code>IntStream mapToInt/ToDouble/ToLong(ToIntFunction<? super T> mapFunction)</code>	I	Come sopra, ma la mappatura è su interi/ecc. (operazione ottimizzata)
<code>Stream<T> sorted()</code>	I	Produce un nuovo stream di elementi ordinati
<code>Stream<T> limit(long k)</code>	I	Limita lo stream a k elementi
<code>Optional<T> max/min(Comparator<? super T> comparator)</code>	T	Restituisce il massimo/minimo elemento all'interno dello stream
<code>T reduce(T identityVal, BinaryOperator<T> accumulator)</code>	T	Effettua l'operazione di riduzione basata sugli elementi dello stream
<code>void forEach(Consumer<? super T> action)</code>	T	Esegue il codice in input su ogni elemento dello Stream
<code><R, A> R collect(Collector<? super T, A, R> collectorFunction)</code>	T	Raccoglie gli elementi di tipo T in un contenitore di tipo R, accumulando in oggetti di tipo A
<code>long count()</code>	T	Conta il numero di elementi

Stream: operazioni intermedie e terminali (2)

- Una volta che uno stream è stato consumato (**operazione terminale**), non può essere riutilizzato
- **Comportamento pigro** (**lazy behavior**): Le operazioni intermedie non vengono eseguite immediatamente, ma solo quando si richiede l'esecuzione di un'operazione terminale

Stream: operazioni intermedie e terminali (3)

- Le operazioni possono essere:
 - **senza stato** (*stateless*): l'elaborazione dei vari elementi può procedere in modo indipendente (es. filter)
 - **con stato** (*stateful*): l'elaborazione di un elemento potrebbe dipendere da quella di altri elementi (es. sorted)



Stream, IntStream, DoubleStream, LongStream

- Poiché **Stream** opera su oggetti, esistono analoghe versioni ottimizzate per lavorare con 3 tipi primitivi:
 - Su int: **IntStream**
 - Su double: **DoubleStream**
 - Su long: **LongStream**
- Tutte queste interfacce estendono l'interfaccia di base **BaseStream**

Come ottenere uno stream?

- 1) Direttamente dai dati: con il metodo statico generico `Stream.of(elenco di dati di un certo tipo)`
- 2) In Java 8 l'interfaccia `Collection` è stata estesa per includere due nuovi metodi di default:
 - `default Stream<E> stream()` – restituisce un nuovo stream sequenziale
 - `default Stream<E> parallelStream()` – restituisce un nuovo stream parallelo, se possibile (altrimenti restituisce uno stream sequenziale)
- 3) E' possibile ottenere uno stream anche per un array, con il metodo statico `Stream<T> Arrays.stream(T[] array)`
- 4) E' possibile ottenere uno stream di righe di testo da `BufferedReader.lines()` oppure da `Files.lines(Path)`
- 5) E' possibile ottenere uno stream di righe anche da `String.lines()`

Stream vs. collection 1

- Lo stream permette di utilizzare uno **stile dichiarativo**
 - Iterazione interna sui dati
- Uno stream non modifica gli elementi dalla collezione di origine

Stream vs. collection 2

- La collection impone l'utilizzo di uno stile imperativo
 - Iterazione esterna sui dati
 - Lo stream si focalizza sulle operazioni di alto livello da eseguire (mattoncini o building blocks) eventualmente anche in parallelo, senza specificare come verranno eseguite

Stream vs. collection 3

- **Stream:** dichiarativo, funzionale, componibile, parallelizzabile

Credits

Le slide di questo corso sono basate sulle slide del Prof. Faralli che sono a loro volta una rielaborazione delle slide del Prof. Navigli

In aggiunta, le slide sono state revisionate dagli studenti borsisti della Facoltà di Ingegneria Informatica, Informatica e Statistica: Mario Marra e Paolo Straniero.

Font ad alta leggibilità Biancoenero® di biancoenero edizioni srl, disegnata da Umberto Mischì, con la consulenza di Alessandra Finzi, Daniele Zanoni e Luciano Perondi (Brevetto n. RM20110000128).

Disponibile gratuitamente per tutte le istituzioni e i privati che la utilizzino per scopi non commerciali.